

Q5. SEATING ARRANGEMENT (CONSTRAINT SATISFACTION / BACKTRACKING PROBLEM)

1. AIM

To develop a backtracking-based solution for seating guests around a round table while satisfying given seating constraints. The solution should ensure that guests who must sit adjacent are placed together, guests who cannot sit together are separated, and all valid seating arrangements are generated without producing invalid arrangements unnecessarily.

2. PROBLEM DESCRIPTION

Guests are seated at a round table. Since the table is circular, the first and last positions are also adjacent. The problem is treated as a Constraint Satisfaction Problem (CSP), where each guest represents a variable and each table position represents a possible value. A backtracking algorithm incrementally assigns guests to seats and immediately checks whether the partial arrangement violates any constraint.

3. SEATING CONSTRAINTS AND RELATIONSHIPS

The following constraints are considered for the sample problem:

- All guests must occupy different seats.
- Alice and Bob must sit adjacent to each other.
- Carol and David must sit adjacent to each other.
- Eve and Frank cannot sit adjacent to each other.
- Bob and Eve cannot sit adjacent to each other.
- Because the table is round, the first and last positions are considered adjacent.
- Rotationally equivalent arrangements are treated as the same by fixing one guest (Alice) at a reference position.

4. CONSTRAINT PRIORITIZATION

Constraints are checked in the following order to reduce invalid arrangements:

1. Check whether the selected seat is already occupied.
2. Check mandatory adjacency constraints involving the newly seated guest.
3. Check forbidden adjacency constraints involving the newly seated guest.
4. When all guests are seated, check circular adjacency between the last and first guests.
5. Verify all mandatory adjacency pairs before accepting the arrangement.

5. ALGORITHM

Backtracking Seating Algorithm:

Step 1: Store the list of guests and the required/forbidden adjacency relationships.

Step 2: Fix one guest, Alice, at the first position to remove duplicate rotations.

Step 3: Place an unseated guest in the next available position.

Step 4: Check the partial arrangement against all applicable constraints.

Step 5: If the placement is valid, recursively place the next guest.

Step 6: If no guest can be placed at the current position, undo the previous placement and try another guest.

Step 7: When all guests are placed, check the circular boundary between the last and first positions.

Step 8: If every constraint is satisfied, record the arrangement as a valid seating plan.

Step 9: Continue backtracking until all possible valid plans have been generated.

6. PSEUDOCODE

BACKTRACK(position):

IF position == number of guests:

IF circular constraints are satisfied:

print seating arrangement

RETURN

FOR each guest not yet seated:

IF guest can be placed at position:

place guest

mark guest as seated

BACKTRACK(position + 1)

unmark guest

remove guest from position

CAN_PLACE(guest, position):

Check required adjacency constraints that can already be evaluated

Check forbidden adjacency constraints with already seated neighbors

RETURN TRUE if no constraint is violated

ELSE RETURN FALSE

7. SAMPLE INPUT

Number of guests = 6

Guests = {Alice, Bob, Carol, David, Eve, Frank}

Must sit adjacent:

(Alice, Bob)

(Carol, David)

Cannot sit adjacent:

(Eve, Frank)

(Bob, Eve)

Table type = Round table

Symmetry breaking:

Alice is fixed at position 1.

8. SAMPLE OUTPUT

```
SEATING ARRANGEMENT USING BACKTRACKING
-----

Valid Seating Plan 1:
Alice -> Bob -> Frank -> Carol -> David -> Eve -> Alice

Valid Seating Plan 2:
Alice -> Bob -> Frank -> David -> Carol -> Eve -> Alice

Valid Seating Plan 3:
Alice -> Eve -> Carol -> David -> Frank -> Bob -> Alice

Valid Seating Plan 4:
Alice -> Eve -> David -> Carol -> Frank -> Bob -> Alice

Total Valid Seating Plans = 4
```

9. CONSTRAINT VALIDATION

Constraint validation is performed during every placement rather than waiting until the complete arrangement is generated. For example, if Eve is placed next to Bob, the partial arrangement is rejected immediately because Bob and Eve cannot sit together. Similarly, placements that make a required adjacent pair impossible are discarded as early as possible. This pruning significantly reduces the number of arrangements explored.

10. TIME COMPLEXITY ANALYSIS

Without pruning, fixing Alice still leaves $(n - 1)!$ possible circular arrangements. Therefore, the worst-case time complexity is $O((n - 1)!)$, with additional constraint-checking overhead depending on the number of constraints. In a general implementation, if each placement checks up to C constraints, the upper bound can be expressed as $O(C \times (n - 1)!)$.

Backtracking is usually much faster in practice because invalid partial arrangements are rejected before all guests are seated. The effectiveness of pruning depends on how restrictive the constraints are and how early they can be checked.

11. SPACE COMPLEXITY

The algorithm requires $O(n)$ space for the current seating arrangement, the visited/seated information, and the recursion stack. If all valid seating plans are stored, additional output space proportional to the number of solutions is required.

12. CORRECTNESS AND COMPLETENESS

Correctness is ensured because an arrangement is accepted only after every mandatory and forbidden adjacency constraint has been validated. A guest is placed only in an unused position, so duplicate seating positions cannot occur.

Completeness is ensured because the backtracking algorithm systematically considers every possible unseated guest for each available position. Whenever a partial arrangement violates a constraint, that branch is safely pruned because no completion of that branch can satisfy the violated constraint. All remaining feasible branches are explored, so every valid seating plan is generated.

13. RESULT

The seating arrangement problem was successfully formulated as a Constraint Satisfaction Problem and solved using backtracking. The algorithm prioritizes restrictive constraints, validates them during partial assignments, prunes invalid branches early, and generates valid circular seating plans. Fixing one guest at a reference position removes rotational duplicates while preserving all unique seating arrangements.